

Лабораторная работа №16

Проектирование иерархии классов. Наследование и полиморфизм

Цель работы:

1. Освоить принципы объектно-ориентированного проектирования иерархий классов
2. Научиться использовать наследование для создания родственных типов
3. Освоить виртуальные методы и позднее связывание (полиморфизм)
4. Изучить абстрактные классы и чисто виртуальные методы
5. Освоить построение диаграмм классов UML для визуализации архитектуры
6. Разработать проектное решение для выбранной предметной области

Теоретическое обоснование

1. Наследование (Inheritance)

Наследование - механизм ООП, позволяющий создавать новый класс на основе существующего, заимствуя его поля и методы и добавляя новые или изменяя существующие.

Отношение «is a»: Производный класс является частным случаем базового.

// Базовый класс (родитель)

```
class Animal {  
protected:  
    string name;  
    int age;  
public:  
    Animal(const string& n, int a) : name(n), age(a) {}  
    virtual void speak() const { cout << "Some sound" << endl; }  
    virtual ~Animal() {}  
};
```

// Производный класс (наследник)

```
class Dog : public Animal {  
private:  
    string breed;  
public:  
    Dog(const string& n, int a, const string& b) : Animal(n, a), breed(b) {}  
};
```

```
void speak() const override { cout << "Woof! Woof!" << endl; }
void wagTail() const { cout << "Tail wagging" << endl; }
};
```

2. Виды наследования в C++

Тип наследования	public	protected	private
public	public → public	protected → protected	private → недоступен
protected	public → protected	protected → protected	private → недоступен
private	public → private	protected → private	private → недоступен

Правило: Отношение «is а» реализуется только через public наследование.

3. Виртуальные методы (Virtual Functions)

Виртуальный метод - метод, который может быть переопределён в производном классе, а вызов через указатель на базовый класс будет направлен к реализации производного класса (позднее связывание).

```
class Base {
public:
    virtual void show() { cout << "Base" << endl; }
    virtual ~Base() {} // Виртуальный деструктор ОБЯЗАТЕЛЕН!
};
```

```
class Derived : public Base {
public:
    void show() override { cout << "Derived" << endl; }
};
```

```
int main() {
    Base* ptr = new Derived();
    ptr->show(); // Вывод: Derived (полиморфизм!)
    delete ptr;
}
```

4. Абстрактные классы (Abstract Classes)

Абстрактный класс - класс, содержащий хотя бы один чисто виртуальный метод. Нельзя создавать объекты абстрактного класса.

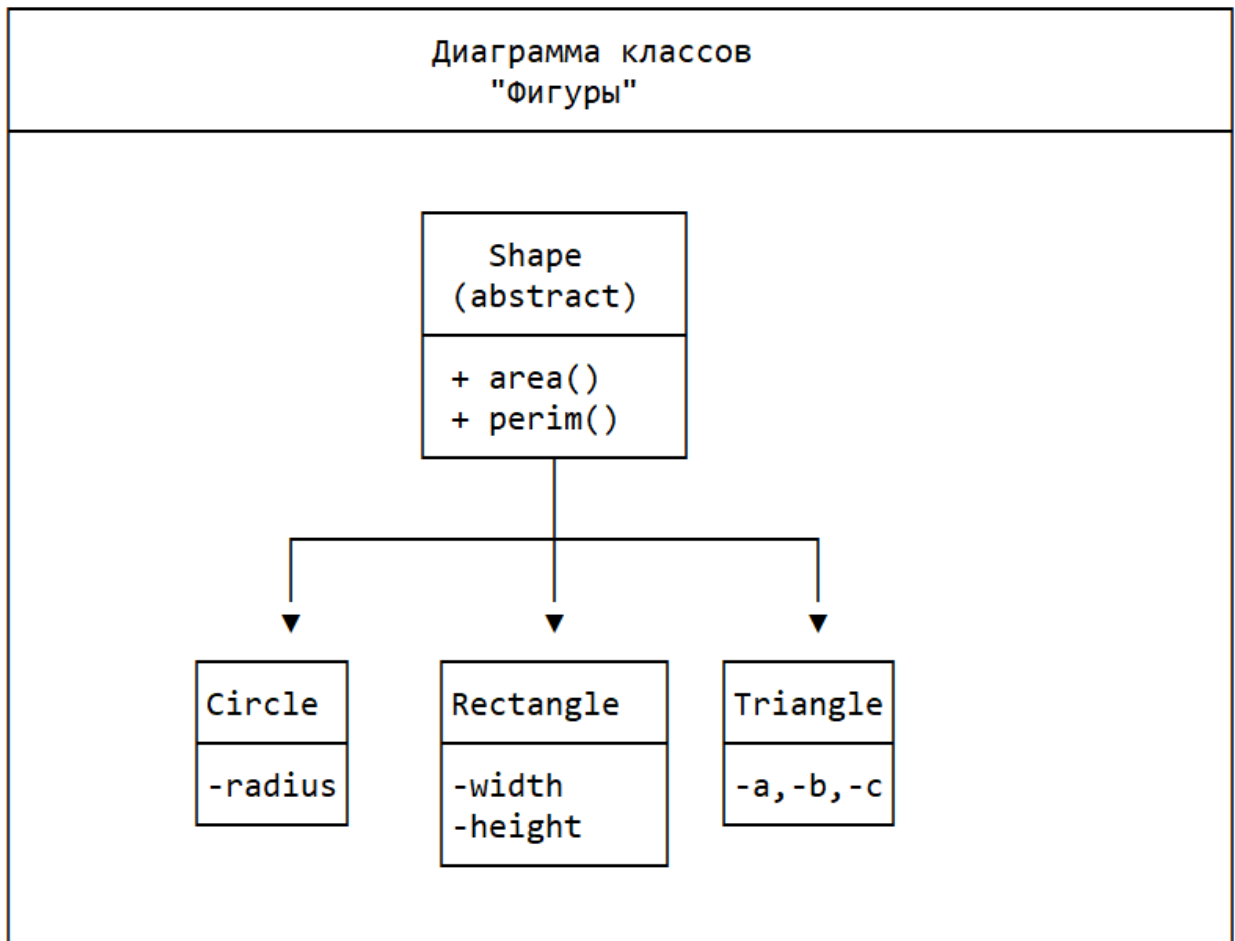
```
class Shape { // Абстрактный класс
public:
    virtual double area() const = 0;    // Чисто виртуальный метод
    virtual double perimeter() const = 0; // Чисто виртуальный метод
    virtual ~Shape() {}
};
```

```
class Circle : public Shape {
private:
    double radius;
public:
    Circle(double r) : radius(r) {}
    double area() const override { return 3.14159 * radius * radius; }
    double perimeter() const override { return 2 * 3.14159 * radius; }
};
```

5. Диаграммы классов UML

Символ	Значение	Пример
▷ (пустая стрелка)	Наследование (is a)	Circle ▷ Shape
◇ (пустой ромб)	Агрегация (has a)	Library ◇ Book
◆ (закрашенный ромб)	Композиция (has a)	House ◆ Room
→ (пунктир)	Зависимость (uses)	Main → Shape

Пример диаграммы классов:



6. Паттерны проектирования (краткое введение)

Паттерн «Стратегия» (Strategy) - инкапсулирует семейство алгоритмов, делая их взаимозаменяемыми.

// Интерфейс стратегии

```
class SortStrategy {
public:
    virtual void sort(vector<int>& data) = 0;
    virtual ~SortStrategy() {}
};
```

// Конкретные стратегии

```
class BubbleSort : public SortStrategy {
public:
    void sort(vector<int>& data) override { /* пузырьковая сортировка */ }
};
```

```
class QuickSort : public SortStrategy {
public:
```

```

    void sort(vector<int>& data) override { /* быстрая сортировка */ }
};

// Контекст
class Sorter {
private:
    unique_ptr<SortStrategy> strategy;
public:
    void setStrategy(unique_ptr<SortStrategy> s) { strategy = move(s); }
    void sort(vector<int>& data) { strategy->sort(data); }
};

```

Пример выполнения практической части

Сквозной пример: Иерархия транспортных средств

Задача: Разработать иерархию классов для представления транспортных средств.

```

#include <iostream>
#include <string>
#include <vector>
#include <memory>

using namespace std;

// === Базовый абстрактный класс ===
class Vehicle {
protected:
    string brand;
    string model;
    int year;
    double speed; // текущая скорость

public:
    Vehicle(const string& b, const string& m, int y)
        : brand(b), model(m), year(y), speed(0) {}

    virtual ~Vehicle() {}

    // Чисто виртуальные методы
    virtual double getMaxSpeed() const = 0;
    virtual double getFuelConsumption() const = 0;

```

```

virtual string getType() const = 0;

// Виртуальные методы с реализацией по умолчанию
virtual void accelerate(double delta) {
    speed += delta;
    if (speed > getMaxSpeed()) speed = getMaxSpeed();
    cout << brand << " " << model << " разогнался до " << speed << "
км/ч" << endl;
}

virtual void brake(double delta) {
    speed -= delta;
    if (speed < 0) speed = 0;
    cout << brand << " " << model << " замедлился до " << speed << "
км/ч" << endl;
}

// Обычный метод
void printInfo() const {
    cout << "=== " << brand << " " << model << " ===" << endl;
    cout << "Год выпуска: " << year << endl;
    cout << "Тип: " << getType() << endl;
    cout << "Макс. скорость: " << getMaxSpeed() << " км/ч" << endl;
    cout << "Расход топлива: " << getFuelConsumption() << " л/100км"
<< endl;
    cout << "Текущая скорость: " << speed << " км/ч" << endl;
}
};

// === Производные классы ===
class Car : public Vehicle {
private:
    int passengerCount;
    double trunkVolume;

public:
    Car(const string& b, const string& m, int y, int passengers, double trunk)
        : Vehicle(b, m, y), passengerCount(passengers), trunkVolume(trunk) {}

    double getMaxSpeed() const override {
        if (brand == "Ferrari") return 350;
        if (brand == "Toyota") return 220;
    }
};

```

```

        return 200;
    }

    double getFuelConsumption() const override {
        if (brand == "Ferrari") return 15.0;
        if (brand == "Toyota") return 8.0;
        return 10.0;
    }

    string getType() const override { return "Легковой автомобиль"; }

    void printPassengerInfo() const {
        cout << "Пассажировместимость: " << passengerCount << " чел." <<
endl
        cout << "Объём багажника: " << trunkVolume << " л" << endl;
    }
};

class Truck : public Vehicle {
private:
    double loadCapacity; // грузоподъёмность (тонн)
    int axles             // количество осей

public:
    Truck(const string& b, const string& m, int y, double capacity, int
axlesCount)
        : Vehicle(b, m, y, loadCapacity(capacity), axles(axlesCount)) {}

    double getMaxSpeed() const override {
        if (loadCapacity > 20) return 90;
        return 110;
    }

    double getFuelConsumption() const override {
        return 25.0 + loadCapacity * 0.5;
    }

    string getType() const override { return "Грузовой автомобиль"; }

    void printLoadInfo() const {
        cout << "Грузоподъёмность: " << loadCapacity << " т" << endl;
        cout << "Количество осей: " << axles << endl;
    }
};

```

```
    }  
};
```

```
class Motorcycle : public Vehicle {
```

```
private:
```

```
    bool hasSidecar;
```

```
public:
```

```
    Motorcycle(const string& b, const string& m, int y, bool sidecar)  
        : Vehicle(b, m, y), hasSidecar(sidecar) {}
```

```
    double getMaxSpeed() const override { return 220; }
```

```
    double getFuelConsumption() const override { return hasSidecar ? 6.0 : 4.5;
```

```
}
```

```
    string getType() const override {  
        return hasSidecar ? "Мотоцикл с коляской" : "Мотоцикл";  
    }
```

```
    void wheelie() const {  
        cout << brand << " " << model << " встаёт на заднее колесо!" << endl;  
    }  
};
```

```
// === Функция, демонстрирующая полиморфизм ===
```

```
void testVehicle(Vehicle* v) {  
    cout << "\n--- Тестирование транспортного средства ---" << endl;  
    v->printInfo();  
    v->accelerate(50);  
    v->brake(20);  
}
```

```
// === Главная функция ===
```

```
int main() {  
    setlocale(LC_ALL, "ru");
```

```
    // Создание объектов разных типов
```

```
    Car ferrari("Ferrari", "F40", 1987, 2, 120);
```

```
    Truck volvo("Volvo", "FH16", 2020, 25.0, 3);
```

```
    Motorcycle harley("Harley-Davidson", "Street Glide", 2019, false);
```

```
// Полиморфное использование через указатель на базовый класс
vector<Vehicle*> vehicles = {&ferrari, &volvo, &harley};
```

```
for (auto v : vehicles) {
    testVehicle(v);
}
```

```
// Вызов специфичных методов (требуется приведение типа)
cout << "\n--- Специфичные методы ---" << endl
ferrari printPassengerInfo();
volvo printLoadInfo();
harley.wheelie();
```

```
return 0;
```

Ожидаемый вывод:

=== Ferrari F40 ===

Год выпуска: 1987

Тип: Легковой автомобиль

Макс. скорость: 350 км/ч

Расход топлива: 15 л/100км

Текущая скорость: 0 км/ч

Ferrari F40 разогнался до 50 км/ч

Ferrari F40 замедлился до 30 км/ч

--- Аналогично для Volvo и Harley-Davidson ---

Индивидуальные задания Базовый уровень (15 вариантов)

Общее задание: Разработать иерархию классов, состоящую из базового абстрактного класса и 2-3 производных классов, согласно варианту. Построить диаграмму классов UML. Продемонстрировать полиморфизм.

№ вар.	Тема	Базовый класс	Поля базового	Производные классы	Специфичные методы
1	Животные	Animal	имя, возраст, вес	Dog, Cat, Bird	bark(), meow(), fly()
2	Сотрудники	Employee	имя, ID, стаж	Manager, Developer, Intern	manage(), code(), learn()

№ вар.	Тема	Базовый класс	Поля базового	Производные классы	Специфи чные методы
3	Фигур ы	Shape	цвет, позиция	Circle, Rectang le, Triangle	area(), perime ter()
4	Трансп орт	Vehicle	марка, модель, год	Car, Truck, Mo torcycle	openTrunk(), loadCargo(), wheelie()
5	Банков ские счета	Account	номер, баланс, владелец	SavingsAccoun t, CheckingAcc ount, CreditAc count	applyInterest(), withdrawFe e(), calculate Credit()
6	Студен ты	Student	имя, курс, группа	UndergradStud ent, GradStude nt, PhdStudent	attendClass(), doResearch() , defendThesi s()
7	Игровы е персона жи	Character	имя, здоровье, уровень	Warrior, Mage, Archer	slash(), castS pell(), shoot()
8	Электр оника	Device	бренд, модель, цена	Laptop, Smartp hone, Tablet	installOS(), makeCall(), s tylusWrite()
9	Домаш няя библио тека	MediaItem	название, год, жанр	Book, DVD, M agazine	read(), watch(), browse()
10	Спорти вные снаряд ы	Equipment	название, вес, цена	Ball, Racket, S kates	bounce(), swi ng(), glide()
11	Музыка льные инстру менты	Instrument	название, тип, цена	Guitar, Piano, Drum	strum(), play Key(), hit()
12	Мебель	Furniture	название, материал, цена	Chair, Table, Wardrobe	sit(), place(), store()

№ вар.	Тема	Базовый класс	Поля базового	Производные классы	Специфичные методы
13	Одежда	Clothing	название, размер, материал	Shirt, Pants, Jacket	wear(), zip(), pocket()
14	Программное обеспечение	Software	название, версия, цена	OS, App, Game	boot(), run(), save()
15	Курсы обучения	Course	название, часы, сложность	Lecture, Lab, Exam	present(), practice(), test()

Средний уровень (15 вариантов)

Общее задание: Разработать иерархию классов с использованием виртуальных методов и абстрактного базового класса. Добавить статическое поле для подсчёта объектов. Реализовать метод клонирования (виртуальный метод clone()). Построить диаграмму UML. Создать фабрику объектов.

№ вар.	Тема	Особенности
1	Животные	Подсчёт объектов каждого вида; метод makeSound()
2	Фигуры	Вычисление площади и периметра; метод scale(factor)
3	Транспорт	Расчёт стоимости топлива; метод fuelCost(distance, price)
4	Банковские счета	Расчёт процентов; метод transfer(Account&, amount)
5	Работники	Расчёт зарплаты (разные системы оплаты); метод calculateBonus()
6	Игровые персонажи	Расчёт урона; метод attack(Character&)

№ вар.	Тема	Особенности
7	Электроника	Расчёт энергопотребления; метод batteryLife()
8	Медиа	Расчёт времени просмотра/чтения; метод getDuration()
9	Спорт	Расчёт калорий; метод burnCalories(time)
10	Еда	Расчёт калорийности; метод cook()
11	Здания	Расчёт площади и объёма; метод buildCost(pricePerSqM)
12	Часы	Расчёт времени до события; метод timeToEvent()
13	Алгоритмы	Реализация разных алгоритмов сортировки (паттерн Стратегия)
14	Шифрование	Реализация разных методов шифрования (паттерн Стратегия)
15	Сжатие	Реализация разных методов сжатия (паттерн Стратегия)

Продвинутый уровень (15 вариантов)

Общее задание: Разработать полноценную мини-систему с использованием наследования, полиморфизма и паттернов проектирования. Реализовать сериализацию (сохранение/загрузку из файла). Создать меню для демонстрации всех возможностей.

№ вар.	Тема	Требования
1	Система управления зоопарком	Учёт животных, кормление, ветеринарные карты

№ вар.	Тема	Требования
2	Система управления банковскими счетами	Разные типы счетов, транзакции, проценты
3	Система управления библиотекой	Книги, журналы, DVD, выдача/возврат
4	Система управления персоналом	Сотрудники, отделы, зарплаты, отчёты
5	Система управления отелем	Номера, бронирование, услуги, расчёт
6	Система управления курсами	Студенты, курсы, оценки, расписание
7	Система управления рестораном	Блюда, заказы, скидки, доставка
8	Система управления спортзалом	Абонементы, тренировки, клиенты, тренеры
9	Система управления кинотеатром	Фильмы, сеансы, залы, билеты
10	Система управления интернет-магазином	Товары, корзина, заказы, оплата
11	Система управления задачами	Задачи, проекты, исполнители, сроки
12	Система управления расписанием	События, напоминания, участники
13	Система управления складом	Товары, поставки, остатки, отгрузка
14	Система управления поликлиникой	Пациенты, врачи, записи, рецепты

№ вар.	Тема	Требования
15	Система управления автосервисом	Автомобили, услуги, заказ-наряды

Контрольные вопросы

1. Что такое наследование? Какие виды наследования есть в C++?
2. Что такое открытое (public) наследование? Какое отношение оно реализует?
3. Что такое виртуальный метод? Для чего он нужен?
4. Что такое чисто виртуальный метод и абстрактный класс?
5. Почему деструктор базового класса должен быть виртуальным?
6. Что такое позднее связывание (динамический полиморфизм)?
7. Как переопределить метод в производном классе? Что такое override?
8. Что такое RTTI? Как работает dynamic_cast?
9. Как вызвать метод базового класса из производного?
10. Что такое диаграмма классов UML? Какие основные элементы?
11. Что такое паттерн «Стратегия»? Где он применяется?
12. Что такое виртуальный деструктор? Почему он важен?

Требования к отчёту (ЛР23)

Титульный лист с названием работы, ФИО студента, номером группы, вариантом

Цель работы

Теоретическая часть (наследование, полиморфизм, UML)

Постановка задачи (согласно варианту)

Диаграмма классов UML (можно от руки или в инструменте)

Листинг программы с комментариями

Результаты работы (скриншоты вывода)

Ответы на контрольные вопросы

Выводы

Ссылка на Git-репозиторий